

CSS Selector Cheat Sheet for Playwright and Selenium

Every CSS selector an automation tester actually uses: the quick reference table of **Playwright-only pseudo-classes** first, then the full standard CSS arsenal, table-locating strategies, and copy-paste examples for Playwright and Selenium.

- Quick reference table
- Playwright pseudo-classes
- Attribute operators
- Table strategies

ON THIS PAGE

1. Quick reference: Playwright pseudo-classes

2. The basics

3. Attribute selectors

4. Combinators

5. Structural pseudo-classes

6. State pseudo-classes

7. Locating in tables

8. Playwright and Selenium usage

9. Pitfalls

1. Quick reference: Playwright pseudo-classes for tables

These are the selectors that make CSS usable on real-world tables. Most are **Playwright-only extensions** (they will not work in `document.querySelector` or Selenium); the standard ones are marked.

PSEUDO-CLASS	SCOPE	USE CASE IN TABLES	STANDARD CSS?
<code>:has(selector)</code>	Descendant match, returns the parent	Row containing a specific cell or button: <code>tr:has(button.delete)</code>	Yes (modern browsers)

<code>:has-text("str")</code>	Own + children's text, substring, case-insensitive	Quick row or cell text search: <code>tr:has-text("Pramod")</code>	No, Playwright only
<code>:text("str")</code>	Smallest element with own text, substring	Cell text match without knowing the tag depth: <code>td :text("Delivered")</code>	No, Playwright only
<code>:text-is("str")</code>	Own text, exact match	Avoid partial-match false positives: <code>:text-is("Total")</code> skips "Subtotal"	No, Playwright only
<code>:text-matches("re", "i")</code>	Own text, regular expression	Pattern-shaped cell values (dates, scores): <code>td:text-matches("\\d+%")</code>	No, Playwright only
<code>:nth-child(n)</code>	DOM position among siblings	Nth row or column overall: <code>tbody tr:nth-child(3)</code> , <code>td:nth-child(2)</code>	Yes
<code>:nth-match(sel, n)</code>	Nth of the whole matched set	Nth <i>matching</i> row, not Nth overall: <code>:nth-match(tr:has-text("Fail"), 2)</code>	No, Playwright only
<code>:not(selector)</code>	Exclusion	Skip header, footer, or total rows: <code>tr:not(.header):not(:has(th))</code>	Yes
<code>:visible</code>	Rendering state	Skip hidden or collapsed rows: <code>tr:visible</code>	No, Playwright only

The one distinction people miss: `:nth-child(3)` counts *all siblings*, while `:nth-match(sel, 3)` counts *only the elements your selector matched*. "Third row that contains Fail" is `:nth-match(tr:has-text("Fail"), 3)` , never `tr:has-text("Fail"):nth-child(3)` .

2. The basics

SELECTOR	MATCHES	EXAMPLE
<code>tag</code>	Every element of that tag	<code>button</code> , <code>input</code> , <code>tr</code>
<code>#id</code>	The element with that id	<code>#login-btn</code>
<code>.class</code>	Elements with that class	<code>.error-message</code>
<code>tag.class</code>	Tag with class (no space!)	<code>button.primary</code>
<code>.a.b</code>	Element with BOTH classes	<code>.btn.disabled</code>
<code>sel1, sel2</code>	Either selector (union)	<code>input, textarea</code>
<code>*</code>	Any element	<code>div > *</code>

3. Attribute selectors

The workhorses for dynamic apps: match on `data-testid` , `name` , `placeholder` , partial ids.

SELECTOR	MEANING	EXAMPLE
<code>[attr]</code>	Attribute exists	<code>[disabled]</code> , <code>[data-testid]</code>
<code>[attr="v"]</code>	Exactly equals	<code>[data-testid="submit"]</code>
<code>[attr^="v"]</code>	Starts with	<code>[id^="user_"]</code> for generated ids
<code>[attr\$="v"]</code>	Ends with	<code>[src\$=".png"]</code>
<code>[attr*="v"]</code>	Contains substring	<code>[class*="error"]</code>
<code>[attr~="v"]</code>	Contains the whole word	<code>[class~="active"]</code> (space-separated list)
<code>[attr = "v"]</code>	Equals or starts with <code>v-</code>	<code>[lang = "en"]</code> matches en, en-US
<code>[attr="v" i]</code>	Case-insensitive match	<code>[placeholder="email" i]</code>

Dynamic id survival kit. Framework-generated ids like `input-4821-field` change every build. Anchor on the stable part: `[id^="input-"][id$="-field"]` , or better, ask

the devs for a `data-testid` and use `[data-testid="email"]`.

4. Combinators

COMBINATOR	RELATIONSHIP	EXAMPLE
<code>A B</code> (space)	B anywhere inside A (any depth)	<code>form input</code>
<code>A > B</code>	B is a direct child of A	<code>ul > li</code> (skips nested lists)
<code>A + B</code>	B immediately after A (same parent)	<code>label + input</code> the input right after its label
<code>A ~ B</code>	Any B after A (same parent)	<code>h2 ~ p</code> every paragraph after the heading

Standard CSS has **no parent or ancestor combinator**: you can walk down, never up.

`:has()` is the exception that fixes it: `div:has(> img)` means "div whose direct child is an img", which effectively selects a parent by its child.

5. Structural pseudo-classes

SELECTOR	MATCHES	TABLE EXAMPLE
<code>:first-child</code> / <code>:last-child</code>	First / last among siblings	<code>tbody tr:first-child</code> , <code>td:last-child</code> (last column)
<code>:nth-child(odd)</code> / <code>(even)</code>	Alternating siblings	<code>tr:nth-child(odd)</code> zebra rows
<code>:nth-child(an+b)</code>	Formula positions	<code>tr:nth-child(3n+1)</code> every third row starting at 1
<code>:nth-last-child(n)</code>	Nth from the end	<code>tr:nth-last-child(2)</code> second-to-last row
<code>:nth-of-type(n)</code>	Nth of that tag only	<code>p:nth-of-type(2)</code> ignores non-p siblings

<code>:only-child</code>	The only sibling	<code>td:only-child</code> a one-cell row (often a "no data" row)
<code>:empty</code>	No children, no text	<code>td:empty</code> blank cells

6. State pseudo-classes

SELECTOR	MATCHES	AUTOMATION USE
<code>:checked</code>	Checked checkbox/radio/option	Assert a row's checkbox is on: <code>tr input:checked</code>
<code>:disabled</code> / <code>:enabled</code>	Form control state	Wait for <code>button[type=submit]:enabled</code>
<code>:focus</code>	The focused element	Keyboard-navigation checks
<code>:required</code> / <code>:optional</code>	Form validation attributes	Count required fields on a form
<code>:placeholder-shown</code>	Input still showing placeholder	Detect untouched fields

7. Locating in tables, end to end

The scenario every interview asks: act on a cell based on another cell in the same row. Given this table:

Order	Customer	Status	Action
#1001	Pramod	Delivered	<button>Delete</button>
#1002	Anita	Pending	<button>Delete</button>
#1003	Rahul	Delivered	<button>Delete</button>

```
// The row that contains "Anita" (standard :has + Playwright :has-text)
tr:has(td:has-text("Anita"))
```

```
// Anita's STATUS cell (3rd column of that row)
tr:has(td:has-text("Anita")) td:nth-child(3)

// Click Delete on Anita's row
tr:has-text("Anita") button

// Second DELIVERED row (matched set, not DOM position)
:nth-match(tr:has-text("Delivered"), 2)

// Every data row except the header, only if visible
tr:not(:has(th)):visible

// Exact status only: "Delivered", never "Not Delivered"
td:text-is("Delivered")
```

8. Using them in Playwright and Selenium

```
// Playwright (TypeScript) - css= prefix is optional
await page.locator('tr:has-text("Anita") button').click();
await expect(page.locator('tr:has(td:has-text("Anita")) td:nth-child(3)'))
  .toHaveText('Pending');

# Selenium (Python) - STANDARD CSS ONLY: no :has-text, :text, :nth-match, :visible
driver.find_element(By.CSS_SELECTOR, 'tr:has(td.customer) button')
# For text-based matching in Selenium, switch to XPath:
driver.find_element(By.XPATH, '//tr[td[text()="Anita"]]//button')
```

Portability warning. `:has-text` , `:text` , `:text-is` , `:text-matches` , `:nth-match` , `:visible` exist only inside Playwright's selector engine. In Selenium, DevTools, or `querySelector` they throw. When you need text matching outside Playwright, use XPath: our [XPath Master Cheat Sheet](#) covers every function and axis.

9. Pitfalls that cost real debugging hours

- **Space vs no space.** `button.primary` (button WITH class) vs `button .primary` (element with class INSIDE a button). One character, opposite meaning.
- **:nth-child counts everything.** A `<style>` or comment-free text node does not count, but ANY element sibling does. If a hidden `<tr class="spacer">` sits in the table, your column math shifts.

- **Classes are not words.** `[class*="act"]` also matches "inactive". Prefer `[class~="active"]` or `.active`.
- **No walking up.** If you find yourself needing "the parent of X", the answer is `:has()`, or XPath's `parent::` axis.
- **Auto-generated class soup.** `.css-1x2y3z` changes every deploy. Anchor on `data-testid`, roles, or stable attributes, and push the team to add them.

Practice all of these against real markup: the [tables practice page](#) and the rest of the [Playwright practice library](#) are built for exactly this. Struggling to find a stable locator at work? [SnapLocator](#) generates and heals them.

Next: the [XPath Master Cheat Sheet](#) →

LEARN THIS LIVE WITH THE TESTING ACADEMY

PLAYWRIGHT COURSE

Playwright Automation Mastery

Master end-to-end Playwright: framework design, API testing, CI, and scale.

[Enroll Now →](#)

AI TESTER BLUEPRINT BATCH

AI Powered Testing Mastery

Become an AI-powered SDET. MCP, RAG, agents, and real projects, live with mentorship.

[Enroll in the Batch →](#)

Built by [The Testing Academy](#) · part of the [Playwright practice library](#).

Ready to go from reading to shipping? Join a live cohort at The Testing Academy.

 [Enroll Now](#)

 [AI Powered Testing Mastery](#)

[Browse all Playwright practice guides →](#)